



Assistant de preuve Coq

David Darras

22/07/2024

1 Les bases

• Remarques :

- L'assistant de preuve Coq est un outil interactif qui permet de faire de la vérification de preuves. On l'utilise en télécommunications, cryptologie, pour les compilateurs, etc. où le développement de programme sûr est centrale.
- On ne peut pas faire de boucle infinie en Coq (**propriété de terminaison uniforme**).
- Le langage de spécification de Coq est appelé **Gallina**.

```

1  (* Définir un type somme (une énumération) *)
2  Inductive pokemon : Set := Bulbasaur | Ivysaur | Venusaur | Charmander | Charmeleon | Charizard.
3  (* Bulbasaur, Ivysaur, Venusaur, etc. sont appelés des constructeurs *)
4  (* On peut mettre Type à la place de Set *)
5  Check pokemon.
6  (*
7  pokemon
8      : Set
9  *)
10 Check Charizard.
11 (*
12 Charizard
13     : pokemon
14 *)
15
16 (* Définir un type somme *)      (* Version OCaml *)
17 Inductive aexp : Set :=      (* type aexp = *)
18 | Cst : nat -> aexp      (* / Cst of int *)
19 | Apl : aexp -> aexp -> aexp      (* / Apl of aexp * aexp *)
20 | Amul : aexp -> aexp -> aexp      (* / Amul of aexp * aexp *)
21 .      (* ;; *)
22 (* En coq, ce sont des fonctions et non des tuples comme en OCaml ! *)
23
24 Inductive nat : Set :=
25 | 0 : nat
26 | S : nat -> nat
27 .
28 (* ou bien *)
29 Inductive nat : Set := 0 | S (n:nat).
30
31 (* Type polymorphe *)
32 Inductive list (A:Type) : Type := (* type 'a list = *)
33 | Nil      (* Nil *)
34 | Cons (a:A) (l:list A).      (* / Cons of 'a * 'a list *)
35
36 Check list.
37
38 (* _ est appelé la variable anonyme, elle couvre tous les autres constructeurs pour le pattern matching *)
39
40 (* Attacher un nom à une expression - c'est comme définir une variable en C *)
41 Definition exp1 := (Amul (Apl (Cst 1) (Cst 2)) (Cst 3)).
42 (* En OCaml : let exp1 = (Amul (Apl (Cst 1) (Cst 2)) (Cst 3)) ;; *)
43

```

```

44 (* Définition locale (comme en OCaml) *)
45 Compute
46   let n := 33 : nat in (* n = 33 *)
47   let e := n + n + n in (* e = 33 + 33 + 33 = 99 *)
48   e + e + e.           (* 99 + 99 + 99 *)
49 (* = 297 : nat *)
50
51 (* Arithmétique des entiers, Peano *)
52
53 (* Condition *)
54 if x then y else z.
55 (* équivalent à *)
56 match x with
57 | True => y
58 | False => z
59 end.
60
61 (* Définir un tuple *)
62 Definition tuple := (13,8,true).
63 (* Check tuple.
64 tuple
65      : nat * nat * bool
66 "*" fait référence au produit cartésien pour des ensembles *)
67
68 (* Définir une fonction non récursive *) (* Version OCaml *)
69 Definition isCst (a:aexp) : bool := (* let isCst (a:aexp) : bool = *)
70   match a with (* match a with *)
71   | Cst _ => true (* Cst _ -> true *)
72   | _ => false (* _ -> false *)
73   end (* *)
74   . (* ;; *)
75
76 Definition add1_or_sub1 (add:bool) (value:nat) : nat :=
77   match add with
78   | true => value + 1
79   | false => value - 1
80   .
81 (* Check add1_or_sub1.
82 add1_or_sub1
83      : bool -> nat -> nat
84 La fonction prend en paramètres un booléen et un entier, et retourne un entier
85 *)
86
87 (* Fonction d'ordre supérieur *)
88 (* Une fonction peut prendre une autre fonction en paramètre *)
89 (* et retourner une fonction *)
90 Definition f2 (f:nat->nat) (x:nat) : nat := f (f x).
91 (* Check f2.
92 f2
93      : (nat -> nat) -> nat -> nat
94 *)
95 Compute (f2 (add1_or_sub1 true) 5).
96 (*
97     = 7
98     : nat
99 *)
100
101 (* Fonction anonyme (= fonction sans nom) *)
102 Compute (fun (x y:nat) => x + y) 1 2.
103 Compute (fun (v:nat*nat) => match v with (x,y) => x + y end) (1,2).
104 (*
105     = 3
106     : nat
107 *)

```

```

108 (* Définir une fonction récursive *)(* Version OCaml *)
109 Fixpoint eval (a:aexp) : nat := (* let rec eval a = *)
110   match a with (* match a with *)
111   | Cst n => n (* | Cst(n) -> n *)
112   | Apl x y => (eval x) + (eval y) (* | Apl(x,y) -> (eval x) + (eval y) *)
113   | Amul x y => (eval x) * (eval y) (* | Amul(x,y) -> (eval x) * (eval y) *)
114   end (* *)
115   . (* ;; *)
116
117 Fixpoint fact (n:nat) : nat :=
118   match n with
119   | 0 => 1
120   | S p => x * fact p end
121 .
122 (* Compute (fact 5).
123    = 120
124    : nat
125 *)
126
127 (* Vérifier qu'une expression est bien formé et obtenir son type *)
128 Check exp1.
129 (*
130  * exp1
131  *      : aexp
132  *)
133
134 (* Afficher la définition d'une expression *)
135 Print exp1.
136 (*
137  * exp1 = Amul (Apl (Cst 1) (Cst 2)) (Cst 3)
138  *      : aexp
139  *)
140
141 (* Evaluer une expression *)
142 Compute (isCst exp1). (* Eval compute in (isCst exp1). *)
143 (*
144  *      = false
145  *      : bool
146  *)
147
148 (* Obtenir la définition d'une notation *)
149 Locate "_ + _".
150 (*
151  Notation "x + y" := (Nat.add x y) : nat_scope (default interpretation)
152  Notation "x + y" := (sum x y) : type_scope
153  *)
154
155 (* Obtenir la définition d'une fonction *)
156 Search "if".
157 (*
158  iff: Prop -> Prop -> Prop
159  Nat.ldiff: nat -> nat -> nat
160  iff_refl: forall A : Prop, A <-> A
161  iff_sym: forall [A B : Prop], A <-> B -> B <-> A
162  ...
163  *)
164
165 (* Importer un module (= des fonctions/types déjà écrites proprement *)
166 Require Import List.
167 (* Remarque: Les listes fonctionnent comme en OCaml *)
168 (* Attention: La concaténation est notée ++ *)

```

- Compute, Print, Check, Section, etc. sont appelés des **commandes**.

2 Les preuves

```

1  (* Le type Prop permet de définir toute logique propositionnelle.
2     Attention: True/False sont de types Prop alors que true/false sont de types bool
3  *)
4  Section Propositional_Logic. (* Permet de définir un bloc de code { ... } comme en C *)
5  Variables P Q R T : Prop.    (* Utile pour restreindre la portée des variables globales *)
6
7  (* Les opérateurs sont :
8     - l'équivalence <->
9     - l'implication ->
10    - la disjonction (OR) \/
11    - la conjonction (AND) /\
12    - la négation (NOT) ~
13  *)
14  Check ((P -> (Q /\ P)) -> (Q -> P)).
15  (* (P → Q → P) → Q → P : Prop *)
16
17  (* Calcul des séquents *)
18  (* https://fr.wikipedia.org/wiki/Calcul_des_s%C3%A9quents *)
19  (* Calcul des prépositions *)
20  (* https://fr.wikipedia.org/wiki/Calcul_des_propositions *)
21
22  (* Pour résoudre des sous buts on utilise des tactics *)
23  (* La preuve est terminée lorsqu'il n'y a plus de sous buts à prouver, il suffit de
24  terminer avec Qed *)
25  (*
26  Hypothèse 1
27  Hypothèse 2
28  .....
29  =====
30  Conclusion (Goal)
31  *)
32  (* L'idée est de partir de la conclusion et de remonter l'arbre jusqu'aux hypothèses qu'on a supposé vrai *)
33
34  (***) TACTICS (***)
35
36  assumption.
37  (* Si le goal correspond à une des hypothèse alors c'est terminé puisque les hypothèses sont supposées vraies *)
38
39  (* Elimination pour l'implication #modus ponens *)
40  apply H.
41  (* Si on a l'hypothèse H : A -> B et qu'on veut montrer B alors il suffit de montrer A *)
42  (* Si dans H0 : A -> B et H1 : A alors apply H1 in H0 donne B as NEW_H *)
43  (*
44  H : A1 -> A2 -> A3 -> A4 -> A
45  ...
46  =====
47  Goal : A
48
49  en plusieurs sous-buts : où il faut montrer A1, A2, A3, A4
50  car si on montre que A1 et A2 et A3 et A4 sont vrais alors A sera vrai
51  *)
52
53  (* Introduction pour l'implication *)
54
55  Section Propositional_Logic.
56  Variables P Q R : Prop.
57  Lemma imp_dist : (P -> (Q -> R)) -> (P -> Q) -> P -> R.
58  Proof. (* Prouvons le lemme imp_dist *)
59  (*
60  1 goal (ID 1)
61
62  P, Q, R : Prop

```

```

63  =====
64  (P -> Q -> R) -> (P -> Q) -> P -> R
65  *)
66  intros H1 H2 H3. (* ou bien : intro H1. intro H2. intro H3 *)
67  (*
68  Montrer : (P -> Q -> R) -> (P -> Q) -> P -> R revient à montrer que R est vrai
69  en supposant les hypothèses (P -> Q -> R) vrai, (P -> Q) vrai et P vrai
70  au quel on donne un nom H1, H2, H3 pour qu'on puisse les utiliser après.
71  *)
72  (*
73  1 goal (ID 4)
74
75  P, Q, R : Prop
76  H1 : P -> Q -> R
77  H2 : P -> Q
78  H3 : P
79  =====
80  R
81  *)
82  apply H1.
83  (*
84  2 goals (ID 5)
85
86  P, Q, R : Prop
87  H1 : P -> Q -> R
88  H2 : P -> Q
89  H3 : P
90  =====
91  P
92
93  goal 2 (ID 6) is:
94  Q
95  *)
96  assumption.
97  (*
98  1 goal (ID 6)
99
100  P, Q, R : Prop
101  H1 : P -> Q -> R
102  H2 : P -> Q
103  H3 : P
104  =====
105  Q
106  *)
107  apply H2.
108  (*
109  1 goal (ID 7)
110
111  P, Q, R : Prop
112  H1 : P -> Q -> R
113  H2 : P -> Q
114  H3 : P
115  =====
116  P
117  *)
118  assumption.
119  (* Plus aucun sous buts à prouver *)
120  Qed.
121  (* La preuve est terminée *)
122
123  Print imp_dist.
124  (* Toute preuve coq peut etre ecrits uniquement à l'aide de fun
125  imp_dist =
126  fun (H1 : P -> Q -> R) (H2 : P -> Q) (H3 : P) => H1 H3 (H2 H3)

```

```

127      : (P -> Q -> R) -> (P -> Q) -> P -> R
128  *)
129
130 Section Imp_trans.
131 Hypothesis H1 : P -> Q. (* introduire une hypothèse *)
132 Hypothesis H2 : Q -> R.
133 Lemma imp_trans: P -> R.
134 Proof.
135   intro H3. (* On suppose P vrai qu'on ajoute au hypothèse et le nouveau sous but est de montrer R vrai *)
136   apply H2.
137   apply H1.
138   assumption.
139 Qed.
140 End Imp_trans.
141
142 trivial.
143 (* Permet de terminer la preuve si le goal est True *)
144
145 (* Il n'y a pas de règle d'introduction pour False
146   mais on peut résoudre le problème avec destruct H.
147   *)
148
149 (* Elimintaion de la conjonction / disjonction *)
150 Section Imp_com.
151
152 Variables P Q : Prop.
153
154 Lemma and_comm : P /\ Q -> Q /\ P.
155 Proof.
156   intro H. (* On ajoute H : P /\ Q aux hypothèses et on veut montrer Q /\ P *)
157   destruct H as [H1 H2]. (* On casse H en deux hypothèses H1 : P et H2 : Q *)
158   split. (* On casse le but en deux sous buts : Montrer Q et P *)
159   - assumption. (* - permet de plonger dans un sous but *)
160   - assumption.
161 Qed.
162
163 Lemma or_comm : P \/ Q -> P \/ Q.
164 Proof.
165   intro H.
166   destruct H as [ H1 | H2 ]. (* Preuve par cas,
167     1er sous but : On suppose que H1 : P, 2eme sous but : On suppose que H2 : Q *)
168   - left. (* On choisit de se focaliser sur le terme de gauche du sous but *)
169     assumption. (* On est arrivé à l'hypothèse H1 : P supposé vrai *)
170   - right. (* On choisit de se focaliser sur le terme de droit du sous but *)
171     assumption. (* On est arrivé à l'hypothèse H2 : Q supposée vraie *)
172 Qed.
173
174 End Imp_com.
175
176 (* Remarque : Il est possible enchaîner les tactics sans faire de pause en remplaçant . par ;
177   Exemple : split;assumption. *)
178 (* Remarque 2 : la tactic tauto. permet de résoudre automatique toutes les propositions tautologiques *)
179
180 (*
181 On appelle prédicat une expression de la forme : A1 -> .... -> An -> Prop avec Ai un Set.
182   *)
183
184 (*
185 Au lieu de définir des variables dans une section on peut utiliser les mots clés :
186 forall et exists dans une proposition
187   *)
188
189 (* règle d'introduction pour le quantificateur universel : pour tout *)
190 intro x. (* Correspond à un soit x - c'est la meme tactic que pour une implciation *)

```

```

191 (* élimination du quantificateur universel *)
192 apply H.
193
194 (* règle d'introduction pour le quantificateur existentiel : il existe *)
195 exists 4.
196 (* élimination *)
197 destruct H as [ x Hx ].
198
199 (* Règle et tactique pour l'égalité *)
200 reflexivity.
201 (* Si dans le goal on a  $X = X$  alors reflexivity termine la preuve *)
202 Lemma Eq : forall x : nat, x = x.
203 Proof.
204   intro x.
205   reflexivity.
206 Qed.
207
208 Goal (1=1 /\ ( 2=2 /\ 3=3 ) ) /\ 4=4.
209 Proof.
210   split. (* Subgoal 1 : (1=1 /\ 2=2 /\ 4=4)   Subgoal 2 : 3=3 *)
211   - (* Subgoal 1 : 1 = 1 /\ 2 = 2 *)
212     split. (* 1 = 1 and 2 = 2 *)
213     + reflexivity. (* 1 = 1 *)
214     + split.
215       * reflexivity. (* 2 = 2 *)
216       * reflexivity. (* 3 = 3 *)
217   - reflexivity. (* 4 = 4 *)
218 Qed.
219
220 (*
221 Pour les bullets (les sous buts : on utilise -, +, *
222 Remarque : On peut enchaîner un même symbole
223 - pour le sous but, -- pour le sous sous but, ---, etc.
224 *)
225
226
227 (* Tactic d'élimination pour l'égalité *)
228 (* Si on a  $H : x = y$  alors *)
229 rewrite -> H.
230 (* transforme tous les x du goal par y *)
231 rewrite <- H.
232 (* transforme tous les y du goal par x *)
233
234 Lemma EqXY3 : forall x y : nat, x = y -> y = 3 -> x = 3.
235 Proof.
236   intros x y H1 H2. (* On pose x, y et on suppose H1 : x = y et H2 : y = 3 vrais.
237                       Montrons que x = 3 *)
238   rewrite <- H2. (* x = y *)
239   assumption. (* On utilise l'hypothèse H3 *)
240 Qed.
241
242 (* Remarque : Il existe aussi symmetry et transitivity. replace, et subst *)
243
244 (* Preuve par induction *)
245 (* Pour rappel une induction ne peut se faire que sur une variable *)
246
247 (* Pour appliquer la définition d'une fonction *)
248 simpl.
249 cbn[foo].
250 (* Pour réaliser des opérations à la main *)
251 Lemma EQ3 : 3 * 2 = 7 - 1.
252 Proof.
253   change (3 * 2) with 6. (* accepté uniquement si c'est une conséquence de la def *)
254   change (7 - 1) with 6.

```

```

255   reflexivity.
256 Qed.
257
258 (* Pour chercher des théorèmes pré-existants *)
259 SearchRewrite (_ + 0).
260 (* plus_0_r: forall n : nat, n + 0 = 0 *)
261 (* On peut ensuite l'utiliser *)
262 rewrite plus_0_r.
263
264 SearchPattern (_ <= _).
265 (*
266 le_n: forall n : nat, n <= n
267 le_0_n: forall n : nat, 0 <= n
268 le_S: forall n m : nat, n <= m -> n <= S m
269 le_pred: forall n m : nat, n <= m -> Nat.pred n <= Nat.pred m
270 le_n_S: forall n m : nat, n <= m -> S n <= S m
271 le_S_n: forall n m : nat, S n <= S m -> n <= m
272 *))
273 apply le_S_n.
274
275 (* tactic exact ? *)
276 exact (eq_refl 42).
277
278 (* Les tactics *)
279 (* Pour les preuves toutes simples, on peut utiliser *)
280 (* Peut faire tout ce que assumptionet reflexivity font *)
281 trivial.
282 (* ou encore mieux : perform a recursive proof search *)
283 auto.
284 (* Si on a une égalité qui est fausse ou une inégalité *)
285 (* Deux constructeur d'un type inductive ne peuvent pas être égal ou false assumption *)
286 discriminate.
287 (* != est noté <> *)
288
289 (* Si une hypothèse est contradictoire avec une autre ou qu'une hypothèse est équivalent à False *)
290 contradiction.
291
292 (* Pour simplifier le but *)
293 simpl.
294 (* Pour simplifier une hypothèse H *)
295 simpl in H.
296
297 (* Pour développer le définiton d'une expression *)
298 unflod.
299
300 (* Dédurre si une égalité est vraie entre deux constructeurs *)
301 inversion.
302
303 (* Casser une hypothèse ET en deux hypothèses *)
304 (* Casser une hypothèse OU en deux sous goals *)
305 (* Génère un sous but par constructor pour un type inductive *)
306 destruct H.
307
308 (* Comme destruct sauf donne une hypothèse d'induction pour les constructors définis récursivement *)
309
310 (* Pour résoudre des additions / multiplications *)
311 ring.
312 (* mieux encore avec division et soustraction en plus *)
313 field.
314 (* Sinon *)
315 tauto.
316
317 (* Tacticals : tactic qui agit sur des tactics *)
318 t1;t2 (* Applique t2 sur tous les sous buts de t1 *)

```



```
319 | t1 || t2 (* Si t1 échoue on essaie t2 *)
320 | repart
321 |
322 |
323 | (* Utiliser un théorème pré-défini *)
324 | apply theorem with ....
325 | (* Si ça ne marche pas, utilisez eapply ou refine *)
326 |
327 | (* Si on a la flemme de finir une preuve *)
328 | Admitted.
329 |
330 | (* Si on a la flemme de finir un sous-but *)
331 | admit.
```